

Лінійна регресія та методи регуляризації

Основи лінійної регресії та підготовка даних

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.metrics import r2_score
```

```
In [2]: def load_data():
    data = fetch_california_housing()
    X = pd.DataFrame(data.data, columns=data.feature_names)
    y = data.target

    return train_test_split(X, y, test_size=0.2, random_state=42)

X_train, X_test, y_train, y_test = load_data()
```

```
In [3]: temp = X_train.copy()
temp["Price"] = y_train
display(temp.head())
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	Price
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03	1.030
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16	3.821
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48	1.726
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11	0.934
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80	0.965

Регуляризація моделей (Ridge та Lasso)

```
In [4]: models = {
    "Linear Regression": LinearRegression(),
    "Ridge (L2)": Ridge(alpha=1.0),
    "Lasso (L1)": Lasso(alpha=0.1)
}

print("--- Оцінка базових моделей (R2 Score) ---")
for name, model in models.items():
    model.fit(X_train, y_train)
    train_score = model.score(X_train, y_train)
    test_score = model.score(X_test, y_test)
    print(f"{name:18}: Train={train_score:.4f}, Test={test_score:.4f}")
```

```
--- Оцінка базових моделей (R2 Score) ---
Linear Regression : Train=0.6126, Test=0.5758
Ridge (L2)       : Train=0.6126, Test=0.5759
Lasso (L1)       : Train=0.5489, Test=0.5318
```

Математичні підходи та поліноміальна регресія

Нормальне рівняння (Normal Equation)

```
In [5]: def solve_normal_equation(X, y):
    X_b = np.c_[np.ones((X.shape[0], 1)), X]

    theta_best = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
    return theta_best

print("--- Матричне розв'язання (Normal Equation) ---")
theta = solve_normal_equation(X_train.values, y_train)
print(f"Intercept (b): {theta[0]:.4f}")
print(f"Перші 3 ваги (w): {theta[1:4]}")
```

```
--- Матричне розв'язання (Normal Equation) ---
Intercept (b): -37.0233
Перші 3 ваги (w): [ 0.44867491  0.00972426 -0.12332334]
```

Поліноміальна регресія

```
In [6]: print("--- Поліноміальна регресія та масштабування (Polynomial Regression and Scaling) ---")

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_poly)
X_test_scaled = scaler.transform(X_test_poly)

poly_model = Ridge(alpha=100)
poly_model.fit(X_train_scaled, y_train)
```

```
print(f"Poly Train Score: {poly_model.score(X_train_scaled, y_train):.4f}")
print(f"Poly Test Score : {poly_model.score(X_test_scaled, y_test):.4f}")
```

```
--- Поліноміальна регресія та масштабування (Polynomial Regression and Scaling) ---
Poly Train Score: 0.6416
Poly Test Score : 0.6117
```